TÀI LIỆU TOÀN KHÓA

Cấu trúc dữ liệu và giải thuật cơ bản

Phạm vi nhập môn. Gồm 6 chương, 12 bài; có mục lục PDF để chuyển đến từng bài. Đọc cùng video tham khảo và làm bài tập thực hành. Bản tổng hợp này không phải chứng chỉ hoặc tài liệu luyện thi được công nhận.

Mục lục

1. Chương 1: Cấu trúc dữ liệu tuyến tính — Mảng và danh sách liên kết
2. Chương 1: Cấu trúc dữ liệu tuyến tính — Ngăn xếp và hàng đợi
3. Chương 2: Tìm kiếm và sắp xếp — Tìm kiếm tuyến tính và nhị phân
4. Chương 2: Tìm kiếm và sắp xếp — Sắp xếp và độ phức tạp
5. Chương 3: Phân tích thuật toán và đệ quy — Big-O, bất biến và kiểm tra thuật toán
6. Chương 3: Phân tích thuật toán và đệ quy — Đệ quy và ngăn xếp lời gọi
7. Chương 4: Tra cứu và dữ liệu có thứ tự — Bảng băm, tập hợp và ánh xạ
8. Chương 4: Tra cứu và dữ liệu có thứ tự — Cây nhị phân và cây tìm kiếm nhị phân
9. Chương 5: Ưu tiên và đồ thị — Heap và hàng đợi ưu tiên
10. Chương 5: Ưu tiên và đồ thị — Đồ thị, BFS và DFS
11. Chương 6: Thiết kế lời giải và thực hành — Chia để trị và quy hoạch động nhập môn
12. Chương 6: Thiết kế lời giải và thực hành — Dự án nhỏ: bộ tra cứu và thống kê dữ liệu

Mảng và danh sách liên kết

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 1: Cấu trúc dữ liệu tuyến tính.
- Các bài trong chương: Mảng và danh sách liên kết; Ngăn xếp và hàng đợi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Chọn cấu trúc phù hợp với thao tác truy cập, tìm kiếm và cập nhật.

Kiến thức nền

Mảng lưu các phần tử ở vùng nhớ liên tiếp trong mô hình cơ bản. Truy cập theo chỉ số có thời gian O(1), nhưng tìm một giá trị trong mảng chưa sắp xếp thường cần O(n). Chèn vào giữa mảng có thể phải dịch chuyển nhiều phần tử nên thường là O(n).

Danh sách liên kết gồm các nút, mỗi nút chứa dữ liệu và liên kết. Danh sách đơn có liên kết tới nút sau; muốn tới phần tử thứ k phải đi từ đầu nên thường O(k). Nếu đã biết nút đứng trước vị trí cần chèn, việc nối một nút mới có thể O(1), chưa tính thời gian tìm vị trí.

Không có cấu trúc luôn tốt nhất: mảng phù hợp truy cập ngẫu nhiên và thường có tính cục bộ bộ nhớ tốt; danh sách liên kết phù hợp một số thao tác nối/tách khi đã có vị trí. Big-O mô tả tốc độ tăng chi phí, không phải số giây chính xác.

1. Mô hình lưu trữ và giới hạn giả định

Trong mô hình mảng cơ bản, phần tử có chỉ số và truy cập trực tiếp. Python list là mảng động các tham chiếu, không hoàn toàn giống mảng số nguyên tĩnh trong C. Vì vậy cần nêu ngôn ngữ và cấu trúc cụ thể khi bàn về bộ nhớ. append của list có thời gian phân bổ bình quân O(1), không có nghĩa mọi lần mở rộng đều hằng số.

Danh sách đơn lưu liên kết next. Biết một nút không đồng nghĩa biết ngay nút trước nó; thao tác xóa cần xét vị trí và mô hình nút. Không khẳng định mọi chèn/xóa danh sách đều O(1) nếu còn phải tìm vị trí.

2. Mô phỏng cập nhật

```
Mảng trước: [A,B,C,D]; chèn X ở chỉ số 2
Dịch D sang phải -> dịch C sang phải -> đặt X
Mảng sau: [A,B,X,C,D]

Danh sách: A -> B -> C -> D
new.next = B.next
B.next = new
Kết quả: A -> B -> X -> C -> D
```

Thứ tự cập nhật liên kết quan trọng: nếu ghi đè B.next mà chưa giữ đường tới C, có thể làm mất phần đuôi khỏi chuỗi truy cập.

3. Ví dụ thực thi Python

```
def insert_copy(values, index, item):
    if not 0 <= index <= len(values):
        raise IndexError("invalid insertion index")
    return values[:index] + [item] + values[index:]

assert insert_copy([1, 3], 1, 2) == [1, 2, 3]
assert insert_copy([], 0, 8) == [8]
```

Hàm tạo bản sao, vì vậy O(n) cả thời gian và bộ nhớ phụ; không nhầm với cập nhật tại chỗ.

4. Bài tập và lời giải

Chọn cấu trúc cho đọc phần tử thứ k rất thường xuyên: mảng là ứng viên hợp lý. Với nối nút khi đã có vị trí, danh sách có lợi thế nhưng vẫn cần quản lý bộ nhớ và liên kết. Nộp bảng truy cập, tìm kiếm, chèn giữa, xóa, kèm giả định; trường hợp rỗng và chỉ một phần tử phải được thử riêng.

> **Bài tập vận dụng bổ sung**
>
> mô phỏng chèn X vào giữa [A,B,C,D], rồi mô phỏng thao tác đó với danh sách liên kết. Đếm thao tác phải thay đổi.

Câu hỏi tự kiểm tra

1. Truy cập phần tử theo chỉ số trong mảng cơ bản có độ phức tạp nào?
2. Tìm một giá trị trong mảng chưa sắp xếp có trường hợp xấu nhất là gì?
3. Danh sách liên kết đơn cơ bản có đặc điểm nào?
4. Chèn vào giữa mảng thường cần thao tác gì?
5. Khi đã biết nút đứng trước, nối một nút mới vào danh sách đơn thường là gì?

Gợi ý đáp án và điểm cần nhớ

- O(1). Địa chỉ phần tử được tính trực tiếp từ chỉ số.
- O(n). Có thể cần kiểm tra tất cả n phần tử.
- Mỗi nút có dữ liệu và liên kết tới nút tiếp theo. Các nút nối với nhau bằng liên kết, không yêu cầu liên tiếp trong bộ nhớ.
- Dịch chuyển các phần tử phía sau. Dịch phần tử tạo chỗ trống nên thường tốn O(n).
- O(1), chưa tính tìm vị trí. Chỉ cần thay đổi một số hữu hạn liên kết.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Ngăn xếp và hàng đợi

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 1: Cấu trúc dữ liệu tuyến tính.
- Các bài trong chương: Mảng và danh sách liên kết; Ngăn xếp và hàng đợi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Mô phỏng LIFO, FIFO và chọn stack hoặc queue đúng tình huống.

Kiến thức nền

Stack là ngăn xếp theo nguyên tắc LIFO: phần tử vào sau được lấy trước. push thêm vào đỉnh, pop lấy khỏi đỉnh, peek xem đỉnh mà không xóa. Với triển khai phù hợp, các thao tác cơ bản có thể O(1). Gọi pop trên stack rỗng cần được xử lý theo quy ước của chương trình.

Queue là hàng đợi theo nguyên tắc FIFO: phần tử vào trước được lấy trước. enqueue thêm vào cuối, dequeue lấy ở đầu. Có thể dùng danh sách liên kết hoặc bộ đệm vòng để tránh dịch toàn bộ phần tử mỗi lần lấy đầu.

Stack hữu ích cho kiểm tra dấu ngoặc và quay lui; queue hữu ích cho xử lý công việc theo thứ tự đến và duyệt BFS. Cấu trúc mô tả quy tắc lấy dữ liệu, không đồng nghĩa với một kiểu lưu bộ nhớ duy nhất.

1. Giao diện trừu tượng trước triển khai

Stack có push, pop, peek, is_empty; queue có enqueue, dequeue, peek. Quy định rõ thao tác trên cấu trúc rỗng: báo lỗi, trả sentinel hoặc một kiểu kết quả riêng. Không dùng None làm dấu lỗi nếu None cũng là dữ liệu hợp lệ mà không có cách phân biệt.

Queue cài bằng list và pop(0) trong Python thường phải dịch phần tử. deque hỗ trợ thao tác ở hai đầu phù hợp hơn cho queue. Đây là lựa chọn triển khai, không thay đổi quy tắc FIFO.

2. Truy vết hai cấu trúc

```
Thao tác        Stack trả      Queue trả
Thêm A          -              -
Thêm B          -              -
```

```
Lấy một         B             A
Thêm C          -             -
Lấy một         C             B
```

Hãy tự ghi phần tử còn lại sau từng dòng. Mô phỏng trên giấy giúp phân biệt quy tắc trước khi viết mã.

3. Kiểm tra dấu ngoặc bằng stack

```
def balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for char in text:
        if char in '([{':
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
    return not stack

assert balanced('([])')
assert not balanced('([)]')
assert not balanced('(')
```

Đây là ví dụ kiểm tra chuỗi dấu ngoặc, không phải bộ phân tích cú pháp Python: ngoặc trong chuỗi ký tự hoặc comment chưa được xử lý theo ngữ pháp ngôn ngữ.

4. Bài tập và lời giải

Thêm test chuỗi rỗng, dấu đóng ở đầu, ba loại ngoặc lồng nhau. Với n ký tự, mỗi ký tự xử lý một lần: thời gian O(n), bộ nhớ tối đa O(n). Đối với hàng đợi công việc, nêu cách ứng xử khi đầy và khi rỗng; không suy ra FIFO đồng nghĩa mọi công việc hoàn tất theo thứ tự nếu nhiều worker chạy đồng thời.

> **Bài tập vận dụng bổ sung**
>
> thực hiện push(A), push(B), pop(), push(C), pop() trên stack. Làm chuỗi tương tự với enqueue và dequeue rồi so sánh kết quả.

Câu hỏi tự kiểm tra

1. Stack tuân theo nguyên tắc nào?
2. Queue tuân theo nguyên tắc nào?
3. push(A), push(B), pop() trên stack trả về gì?
4. Thao tác peek trên stack thường làm gì?
5. Thuật toán BFS thường dùng cấu trúc nào?

Gợi ý đáp án và điểm cần nhớ

- LIFO. LIFO nghĩa là vào sau, ra trước.
- FIFO. FIFO nghĩa là vào trước, ra trước.
- B. B được đưa vào sau nên nằm trên đỉnh.
- Xem đỉnh mà không xóa. peek đọc phần tử trên đỉnh nhưng giữ nguyên stack.
- Queue. Queue giúp xử lý các đỉnh theo thứ tự khám phá từng lớp.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Tìm kiếm tuyến tính và nhị phân

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 2: Tìm kiếm và sắp xếp.
- Các bài trong chương: Tìm kiếm tuyến tính và nhị phân; Sắp xếp và độ phức tạp.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Xác định điều kiện áp dụng và so sánh chi phí tìm kiếm.

Kiến thức nền

Tìm kiếm tuyến tính kiểm tra lần lượt các phần tử; dùng được với dữ liệu chưa sắp xếp. Trường hợp xấu nhất phải xét n phần tử nên là O(n). Kết quả tìm kiếm nên quy ước rõ vị trí tìm thấy hoặc dấu hiệu không có dữ liệu.

Tìm kiếm nhị phân trên mảng đã sắp xếp so sánh mục tiêu với phần tử giữa rồi bỏ một nửa khoảng tìm kiếm. Số bước so sánh trong trường hợp xấu nhất là O(log n). Điều kiện sắp xếp phải phù hợp phép so sánh; không được áp dụng trực tiếp lên một mảng tùy ý chưa sắp xếp.

Trong biến thể dùng hai biên đóng, nếu mục tiêu nhỏ hơn phần tử giữa thì cập nhật biên phải thành mid-1; nếu lớn hơn thì biên trái thành mid+1. Cần bảo đảm khoảng tìm kiếm giảm sau mỗi bước để tránh vòng lặp vô hạn.

1. Hợp đồng hàm tìm kiếm

Quy định trả chỉ số hay giá trị, và trả gì khi không tìm thấy. Nếu dữ liệu có phần tử trùng, yêu cầu có thể là một vị trí bất kỳ, vị trí đầu hoặc cuối; đó là ba hợp đồng khác nhau. Tìm nhị phân cần mảng đã sắp theo cùng phép so sánh đang dùng.

2. Cài đặt có biên rõ ràng

```
def binary_search(values, target):
    left, right = 0, len(values) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if values[mid] == target:
            return mid
```

```
        if values[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

assert binary_search([], 3) == -1
assert binary_search([2, 5, 8, 13, 21], 13) == 3
assert binary_search([2, 5, 8, 13, 21], 7) == -1
```

Bất biến: nếu target tồn tại ở phần chưa loại thì nó nằm trong khoảng đóng left..right. mid±1 bảo đảm khoảng giảm. Không trộn công thức biên đóng này với thuật toán khoảng nửa mở mà không sửa điều kiện tương ứng.

3. Truy vết và phân tích

Trong [2,5,8,13,21], tìm 13: mid=2 có 8 nên left=3; mid=3 có 13 nên trả 3. Khoảng giảm một nửa nên O(log n) lần so sánh trên mảng truy cập O(1). Nếu phải sắp xếp đầu vào cho chỉ một lần tìm, cần tính thêm chi phí sắp xếp.

4. Bài tập mở rộng có đáp án định hướng

Sửa hàm để trả vị trí đầu của 5 trong [1,5,5,5,9]. Khi gặp bằng, lưu ứng viên và tiếp tục tìm bên trái; không trả ngay. Kiểm thử đầu, cuối, rỗng, một phần tử và tất cả bằng nhau. Giải thích vì sao đổi left=mid có thể làm lặp vô hạn khi left sát right.

> **Bài tập vận dụng bổ sung**
>
> tìm 13 trong [2,5,8,13,21,34,55]. Ghi hai biên và phần tử giữa ở từng bước. Thử tiếp một giá trị không có trong mảng.

Câu hỏi tự kiểm tra

1. Tìm kiếm nhị phân trực tiếp trên mảng cần điều kiện nào?
2. Trường hợp xấu nhất của tìm kiếm tuyến tính là gì?
3. Trường hợp xấu nhất của tìm kiếm nhị phân trên mảng là gì?
4. Nếu mục tiêu nhỏ hơn phần tử giữa của mảng tăng dần, nên xét đâu?
5. Vì sao phải cập nhật biên sau mỗi bước tìm kiếm nhị phân?

Gợi ý đáp án và điểm cần nhớ

- Mảng đã sắp xếp theo phép so sánh sử dụng. Thứ tự cho phép loại bỏ một nửa không gian tìm kiếm.
- O(n). Có thể phải đọc hết mảng trước khi kết luận.
- O(log n). Mỗi bước giảm khoảng tìm kiếm xuống khoảng một nửa.
- Nửa bên trái. Mọi phần tử bên phải giữa đều không nhỏ hơn phần tử giữa.

- Để khoảng tìm kiếm giảm và thuật toán kết thúc. Nếu biên không tiến triển, thuật toán có thể lặp vô hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Sắp xếp và độ phức tạp

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 2: Tìm kiếm và sắp xếp.
- Các bài trong chương: Tìm kiếm tuyến tính và nhị phân; Sắp xếp và độ phức tạp.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> ### Mục tiêu bài học
>
> Đọc Big-O và mô tả hai thuật toán sắp xếp cơ bản.

Kiến thức nền

Big-O mô tả một chặn trên tiệm cận của chi phí theo kích thước đầu vào. O(n) tăng tuyến tính, $O(n^2)$ tăng theo bình phương. Khi so sánh cần cùng loại chi phí và trường hợp phân tích; không suy ra thời gian thực tế chỉ từ Big-O.

Selection sort chọn phần tử nhỏ nhất trong phần chưa sắp xếp rồi đưa về đầu phần đó. Thuật toán thường có $O(n^2)$ phép so sánh và có thể thực hiện tại chỗ. Merge sort chia dữ liệu, sắp xếp các nửa rồi trộn hai dãy đã có thứ tự; thời gian thường O(n log n).

Bản merge sort thông thường trên mảng cần O(n) bộ nhớ phụ. Một thuật toán ổn định giữ thứ tự tương đối của các phần tử có khóa bằng nhau. Ổn định và tại chỗ là hai thuộc tính khác nhau, không phải tên gọi của độ phức tạp thời gian.

1. Đặt tiêu chí trước khi chọn thuật toán

Đầu ra tăng hay giảm, có giữ phần tử bằng nhau theo thứ tự cũ không, có sửa đầu vào không, giới hạn bộ nhớ bao nhiêu? Một thuật toán tại chỗ và một thuật toán ổn định có thể phù hợp các nhu cầu khác nhau. So sánh chi phí phải cùng mô hình dữ liệu.

2. Merge sort đầy đủ cho danh sách số

```
def merge_sort(values):
    if len(values) <= 1:
        return values[:]
    middle = len(values) // 2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
```

```
    output = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            output.append(left[i]); i += 1
        else:
            output.append(right[j]); j += 1
    return output + left[i:] + right[j:]

assert merge_sort([4, 2, 5, 1]) == [1, 2, 4, 5]
assert merge_sort([2, 2, -1]) == [-1, 2, 2]
```

Chọn bên trái khi khóa bằng giúp giữ ổn định nếu phép so sánh theo khóa nhất quán. Bản ví dụ dùng slicing và danh sách phụ để dễ hiểu, không phải bản tối ưu bộ nhớ.

3. Vì sao O(n log n)

Chia đôi tạo khoảng log n mức; công việc trộn trên mỗi mức là O(n). Không ghi riêng một lần trộn rồi kết luận toàn thuật toán O(n). Với bản cài thông thường, bộ nhớ phụ cực đại là O(n) cộng ngăn xếp O(log n), vẫn O(n).

4. Bài tập và lời giải

Trộn [1,4,7] với [2,3,8] thu [1,2,3,4,7,8]; ở mỗi bước chọn đầu nhỏ hơn chưa dùng. Kiểm tra đầu ra vừa có thứ tự vừa giữ số lần xuất hiện từng phần tử. Một chương trình trả [] có thứ tự nhưng đã mất dữ liệu, nên chỉ kiểm tra tăng dần là chưa đủ.

> **Bài tập vận dụng bổ sung**
>
> mô phỏng hai bước selection sort cho [4,2,5,1]. Trộn [1,4,7] với [2,3,8]. Với bản ghi cùng điểm, giải thích vì sao giữ thứ tự ban đầu có thể hữu ích.

Câu hỏi tự kiểm tra

1. $O(n^2)$ thường mô tả mức tăng chi phí như thế nào?
2. Selection sort thường chọn gì ở mỗi bước?
3. Thời gian thông thường của merge sort là gì?
4. Sắp xếp ổn định có ý nghĩa gì?
5. Merge sort thông thường trên mảng thường cần bộ nhớ phụ bao nhiêu?

Gợi ý đáp án và điểm cần nhớ

- Theo bình phương kích thước đầu vào. Big-O biểu diễn quan hệ tăng trưởng, không phải thời gian tuyệt đối.
- Phần tử nhỏ nhất trong phần chưa sắp xếp. Phần tử nhỏ nhất được đưa về đầu phần chưa sắp xếp.

- O(n log n). Có khoảng log n mức chia và O(n) công việc trộn mỗi mức.
- Giữ thứ tự tương đối của phần tử có khóa bằng nhau. Tính ổn định xét các phần tử có khóa so sánh bằng nhau.
- O(n). Bộ đệm trộn thông thường có kích thước tỉ lệ với số phần tử.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Big-O, bất biến và kiểm tra thuật toán

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 3: Phân tích thuật toán và đệ quy.
- Các bài trong chương: Big-O, bất biến và kiểm tra thuật toán; Đệ quy và ngăn xếp lời gọi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Phân tích chi phí theo kích thước đầu vào và nêu điều kiện để thuật toán đúng.

Kiến thức nền

Kích thước đầu vào n cần được xác định trước khi phân tích. Một vòng lặp thăm n phần tử thường O(n); hai vòng lặp lồng nhau, mỗi vòng n bước, thường $O(n^2)$. Hai vòng nối tiếp có chi phí O(n)+O(n), vẫn là O(n). Vòng lặp chia đôi miền mỗi bước thường gợi ý O(log n).

Phân biệt trường hợp tốt nhất, xấu nhất và trung bình. Big-O là chặn trên tiệm cận; hằng số, kiểu dữ liệu và môi trường vẫn ảnh hưởng thời gian chạy thực tế. Bộ nhớ phụ phân tích riêng với bộ nhớ chứa đầu vào.

Bất biến là mệnh đề đúng tại các mốc trong vòng lặp, giúp lập luận đúng đắn. Khi tìm số lớn nhất, sau k bước, biến best phải chứa giá trị lớn nhất trong k phần tử đã xét. Test rỗng, một phần tử, trùng lặp và biên giúp tìm lỗi; một vài test qua chưa chứng minh đúng với mọi đầu vào.

1. Đếm trước khi rút gọn

```
for i in range(n): pass          -> n bước
Hai vòng trên nối tiếp           -> 2n bước -> O(n)
Hai vòng n bước lồng nhau        -> n*n bước -> O(n²)
while n > 1: n //= 2             -> khoảng log2(n) bước
```

Nếu vòng trong không chạy n lần mà phụ thuộc i, phải viết tổng thực tế. Với j từ 0 tới i-1, số bước là n(n-1)/2, vẫn $O(n^2)$. Không suy ra cứ thấy hai vòng là chắc $O(n^2)$.

2. Lập luận bằng bất biến

```
def maximum(values):
    if not values:
        raise ValueError('empty input')
    best = values[0]
    for item in values[1:]:
        if item > best:
            best = item
    return best

assert maximum([-9, -2, -5]) == -2
assert maximum([7]) == 7
```

Khởi tạo: best đúng cho tiền tố dài một. Duy trì: so sánh phần tử mới giữ max của tiền tố đã xét. Kết thúc: tiền tố là toàn mảng. Bản dùng slicing ở ví dụ tạo danh sách phụ; nếu phân tích bộ nhớ phải tính chi phí này hoặc chuyển sang lặp chỉ số.

3. Thiết kế test theo đặc tính

Kết quả max phải thuộc dữ liệu và không nhỏ hơn mọi phần tử. Các đặc tính giúp chọn test rỗng, âm, lặp, tăng và giảm. Test chạy qua không thay thế chứng minh; chứng minh cũng không bảo đảm cài đặt không có lỗi gõ.

4. Bài tập và đáp án

Tính tổng từ i=0 đến n-1 của i: n(n-1)/2. Giải thích khác nhau giữa O(n), Θ(n) và một phép đo giây: chặn trên, chặn khít tiệm cận và kết quả thực nghiệm. Nếu chỉ biết thuật toán $O(n^2)$, không tự kết luận đó là chặn khít tốt nhất.

> ### Bài tập vận dụng bổ sung
>
> Phân tích hai vòng nối tiếp và hai vòng lồng nhau. Viết hàm tìm max có quy ước khi đầu vào rỗng; thử [-5,-2,-9], [7], [2,2].

Hướng giải bài tập bổ sung

Chi phí lần lượt O(n) và $O(n^2)$. Với mảng rỗng phải trả dấu hiệu không có kết quả hoặc báo lỗi đã quy định, không tự trả 0.

Câu hỏi tự kiểm tra

1. Hai vòng n bước chạy nối tiếp thường có Big-O nào?
2. Hai vòng n bước lồng nhau thường có chi phí nào?
3. Bộ nhớ phụ nên được đánh giá thế nào?
4. Khởi tạo max bằng 0 dễ sai với dữ liệu nào?

5. Bất biến vòng lặp dùng để làm gì?

Gợi ý đáp án và điểm cần nhớ

- O(n). Hệ số 2 được bỏ trong biểu diễn tiệm cận.
- $O(n^2)$. Tổng số lần thực hiện phần thân thường là n nhân n.
- Riêng với chi phí thời gian. Thuật toán có thể nhanh hơn nhưng dùng nhiều bộ nhớ hơn.
- Mảng chỉ có số âm. 0 không thuộc mảng và lớn hơn mọi phần tử âm.
- Hỗ trợ lập luận trạng thái và tính đúng đắn. Bất biến mô tả điều được giữ đúng trong quá trình lặp.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Đệ quy và ngăn xếp lời gọi

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 3: Phân tích thuật toán và đệ quy.
- Các bài trong chương: Big-O, bất biến và kiểm tra thuật toán; Đệ quy và ngăn xếp lời gọi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Xác định trường hợp cơ sở, bước tiến triển và chi phí của lời giải đệ quy.

Kiến thức nền

Hàm đệ quy gọi lại chính nó để giải bài toán nhỏ hơn. Cần có trường hợp cơ sở và đại lượng giảm tới trường hợp đó. Với giai thừa n không âm: 0! = 1 và n! = n*(n-1)!. Nếu không kiểm tra miền đầu vào, lời gọi với số âm có thể không bao giờ tới cơ sở.

Mỗi lời gọi lưu trạng thái trên ngăn xếp lời gọi. Tính giai thừa theo cách trên có n bước và độ sâu O(n). Đệ quy không tự làm thuật toán nhanh hơn; phiên bản lặp có thể giảm bộ nhớ phụ.

Một bài toán chia thành nhiều nhánh có thể lặp lại tính toán. Fibonacci đệ quy trực tiếp tính nhiều lần cùng giá trị; memoization lưu lời giải bài toán con để tái sử dụng. Phải phân biệt số lời gọi với độ sâu ngăn xếp. Trong thực tế, giới hạn ngăn xếp có thể làm đệ quy sâu thất bại.

1. Ba câu hỏi bắt buộc

Đầu vào nào hợp lệ? Khi nào giải trực tiếp? Mỗi lời gọi tiến gần cơ sở bằng cách nào? Thiếu một câu trả lời có thể gây không kết thúc hoặc xử lý sai biên. Không đặt cơ sở theo cảm tính chỉ để chương trình dừng.

2. Giai thừa và điều kiện miền

```
def factorial(n):
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError('n must be a non-negative integer')
    def solve(k):
        if k == 0:
            return 1
        return k * solve(k - 1)
    return solve(n)
```

```
assert factorial(0) == 1
assert factorial(5) == 120
```

Ví dụ dùng số nhỏ để học; Python giới hạn độ sâu đệ quy. Với dữ liệu lớn nên lựa chọn cách lặp hoặc hàm thư viện phù hợp, không tự tăng giới hạn ngăn xếp để che thiết kế.

3. Truy vết thời gian và bộ nhớ

factorial(3) tạo các lời gọi 3,2,1,0 rồi trả ngược. Số phép nhân là ba, ngăn xếp sâu tuyến tính. Với Fibonacci trực tiếp, cây lời gọi có các nhánh lặp; độ sâu không bằng tổng số nút trong cây. Memoization giảm tính lại bằng bộ nhớ lưu kết quả.

4. Bài tập có hướng giải

Viết tổng danh sách bằng đệ quy với chỉ số thay vì cắt danh sách ở mỗi bước. Cơ sở index==len(values) trả 0; bước tiếp cộng values[index]. Phân tích O(n) thời gian và O(n) ngăn xếp trong mô hình phép cộng hằng số. Kiểm thử danh sách rỗng, âm và một phần tử; ghi rõ giả định về kích thước số nguyên nếu phân tích chi phí bit.

> **Bài tập vận dụng bổ sung**
>
> Viết giả mã giai thừa có kiểm tra n>=0. Vẽ cây lời gọi Fibonacci(5) và đánh dấu các giá trị được tính lại; đề xuất cách lưu kết quả.

Hướng giải bài tập bổ sung

Có cơ sở, miền đầu vào và bước tiến triển rõ ràng. Memoization tránh tính lại, còn vòng lặp có thể tránh ngăn xếp sâu cho giai thừa.

Câu hỏi tự kiểm tra

1. Đệ quy kết thúc thường cần gì?
2. Giá trị 0! theo định nghĩa thông thường là gì?
3. Ngăn xếp lời gọi lưu gì?
4. Memoization giúp gì?
5. Đệ quy có luôn nhanh hơn lặp không?

Gợi ý đáp án và điểm cần nhớ

- Cơ sở và bước tiến tới cơ sở. Bài toán phải giảm tới tình huống giải trực tiếp.
- 1. 0! = 1 làm cơ sở cho công thức giai thừa.
- Trạng thái các lời gọi chưa hoàn tất. Lời gọi con trả về để tiếp tục lời gọi cha.
- Tái sử dụng kết quả bài toán con. Các trạng thái đã giải được lưu lại.

- Không. Thời gian và bộ nhớ phụ thuộc cách phân rã và triển khai.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Bảng băm, tập hợp và ánh xạ

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 4: Tra cứu và dữ liệu có thứ tự.
- Các bài trong chương: Bảng băm, tập hợp và ánh xạ; Cây nhị phân và cây tìm kiếm nhị phân.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Dùng ánh xạ đếm tần suất và giải thích va chạm băm.

Kiến thức nền

Bảng băm biến khóa thành vị trí tra cứu thông qua hàm băm. Map lưu cặp khóa–giá trị; set lưu các phần tử phân biệt. Khi các khóa khác nhau có cùng vị trí băm, cấu trúc phải xử lý va chạm, chẳng hạn nối chuỗi hoặc địa chỉ mở.

Tra cứu, chèn và xóa thường có thời gian kỳ vọng O(1) trong điều kiện thích hợp; không phải bảo đảm xấu nhất O(1). Số lượng phần tử, số ô và chất lượng hàm băm ảnh hưởng hiệu năng. Khóa dùng trong bảng băm cần có tính băm và so sánh nhất quán theo ngôn ngữ.

Đếm từ bằng map tăng bộ đếm mỗi khi gặp từ. Nếu chuẩn hóa chữ hoa/thường, phải nêu quy tắc trước. Dùng set loại bản sao nhưng không tự tạo thứ tự sắp xếp. Nếu bài toán cần duyệt tăng dần, cần bước sắp xếp hoặc cấu trúc có thứ tự thích hợp.

1. Từ khóa tới vị trí lưu

Hàm băm và số ô bảng quyết định vị trí khởi đầu của khóa. Va chạm không có nghĩa hai khóa bằng nhau. Cấu trúc vẫn cần phép so sánh khóa đúng. Hệ số tải là số phần tử chia số ô theo mô hình bảng; khi tăng quá mức có thể cần mở rộng và băm lại tùy triển khai.

2. Bộ đếm có quy tắc đầu vào rõ

```
def frequencies(words, ignore_case=False):
    result = {}
    for word in words:
        key = word.casefold() if ignore_case else word
        result[key] = result.get(key, 0) + 1
    return result
```

```
assert frequencies(['An', 'an']) == {'An': 1, 'an': 1}
assert frequencies(['An', 'an'], True) == {'an': 2}
```

casefold là một lựa chọn xử lý chữ hoa/thường, không tự giải quyết mọi khác biệt biểu diễn Unicode. Nếu ứng dụng cần chuẩn hóa NFC, phải nêu thêm bước đó và viết test tương ứng.

3. Độ phức tạp và giới hạn

Trong điều kiện băm tốt, n cập nhật có thời gian kỳ vọng O(n), bộ nhớ O(k) cho k khóa khác nhau, chưa tính độ dài khóa và chi phí băm chuỗi. Không khẳng định chuỗi rất dài luôn được xử lý cùng chi phí như một ký tự.

Set có thể kiểm tra trùng, nhưng sau khi loại bản sao cần quyết định thứ tự đầu ra. Không dựa vào một lần chạy để suy ra set có thứ tự sắp xếp.

4. Bài tập và đáp án

Với An,Bình,An,an và ignore_case=True, kết quả là an:3,bình:1. Hãy thêm báo cáo giảm dần theo tần suất rồi tăng theo tên khi hòa; bước sắp xếp k khóa có chi phí thêm O(k log k). Đối chiếu tổng đếm với bốn đầu vào để phát hiện mất hoặc nhân đôi dữ liệu.

> **Bài tập vận dụng bổ sung**
>
> Viết bộ đếm ký tự và bài toán kiểm tra có phần tử trùng. Kiểm thử dữ liệu rỗng, tất cả giống nhau và phân biệt chữ A với a theo quy tắc đã chọn.

Hướng giải bài tập bổ sung

Dùng set để phát hiện trùng, map để lưu tần suất. Giải thích thời gian kỳ vọng O(n), bộ nhớ O(k) với k khóa phân biệt.

Câu hỏi tự kiểm tra

1. Map lưu loại dữ liệu nào?
2. Set dùng chủ yếu để biểu diễn gì?
3. Va chạm băm xảy ra khi nào?
4. O(1) của bảng băm thường cần hiểu ra sao?
5. Với a,b,a,a thì tần suất a là bao nhiêu?

Gợi ý đáp án và điểm cần nhớ

- Cặp khóa–giá trị. Map liên kết một khóa với một giá trị.
- Các phần tử phân biệt. Set biểu diễn tính thuộc tập và tính duy nhất.
- Khóa khác nhau ánh xạ tới cùng vị trí. Cần chiến lược xử lý nhiều khóa chung vị trí.

- Hiệu năng kỳ vọng với điều kiện phù hợp. Phân bố khóa và triển khai ảnh hưởng chi phí.
- 3. Map tăng bộ đếm cho mỗi lần xuất hiện của a.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Cây nhị phân và cây tìm kiếm nhị phân

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 4: Tra cứu và dữ liệu có thứ tự.
- Các bài trong chương: Bảng băm, tập hợp và ánh xạ; Cây nhị phân và cây tìm kiếm nhị phân.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Phân biệt cây nhị phân với BST; mô phỏng tìm kiếm và duyệt inorder.

Kiến thức nền

Cây có các nút và quan hệ cha–con. Cây nhị phân có tối đa hai con tại mỗi nút. BST bổ sung quy tắc thứ tự; trong phiên bản không có khóa trùng, mọi khóa bên trái nhỏ hơn nút và bên phải lớn hơn nút. Có hai con không đủ để kết luận là BST.

Tìm kiếm so sánh mục tiêu với khóa nút hiện tại rồi chọn một nhánh. Chi phí phụ thuộc chiều cao h, thường O(h). Nếu cây cân đối thì h là O(log n); nếu chèn khóa tăng dần vào BST không tự cân bằng, cây có thể lệch và tìm kiếm tới O(n).

Duyệt inorder đi trái, nút, phải; với BST hợp lệ cho dãy tăng dần. Kiểm tra BST phải xét ràng buộc toàn cây con, không chỉ hai nút con trực tiếp. Bài này giới thiệu nguyên lý; thuật toán tự cân bằng như AVL không thuộc yêu cầu thực hành bắt buộc.

1. Các khái niệm cần thống nhất

Gốc không có cha; lá không có con. Chiều cao có thể tính bằng số cạnh hoặc số nút tùy tài liệu; phải ghi quy ước. Bài này dùng chiều cao theo số cạnh khi mô tả sơ đồ, còn phân tích O(h) không thay đổi ý nghĩa tiệm cận.

2. Tìm kiếm BST không dùng đệ quy

```
class Node:
    def __init__(self, key, left=None, right=None):
        self.key, self.left, self.right = key, left, right

def contains(root, target):
    node = root
```

```
    while node is not None:
        if target == node.key:
            return True
        node = node.left if target < node.key else node.right
    return False

root = Node(4, Node(2, Node(1), Node(3)), Node(6))
assert contains(root, 3)
assert not contains(root, 5)
```

Thuật toán dùng bộ nhớ phụ O(1) cho tìm kiếm lặp; chi phí thời gian phụ thuộc chiều cao. Không lấy chiều cao của một hình cân đối để kết luận mọi cây đều log n.

3. Kiểm tra thứ tự toàn cây

Gốc 10, con trái 5, nhưng trong cây con trái có 12: mỗi cặp cha-con cục bộ có thể trông hợp lý ở một số cạnh nhưng 12 vẫn vi phạm ràng buộc nhỏ hơn 10. Kiểm tra hợp lệ cần truyền khoảng cho toàn cây con hoặc xem inorder theo chính sách khóa trùng.

4. Bài tập có hướng giải

Thêm thao tác chèn không cho khóa trùng; khi gặp bằng thì giữ nguyên hoặc báo lỗi theo hợp đồng. Viết test cây rỗng, khóa ở gốc, hai nhánh và chuỗi lệch. So sánh thứ tự chèn tăng dần với thứ tự tạo cây cân đối; không thay đổi bộ dữ liệu giữa hai phép so sánh.

> **Bài tập vận dụng bổ sung**
>
> Vẽ BST khi chèn 1,2,3,4 và so sánh chiều cao với thứ tự 3,1,4,2. Viết truy vết tìm một khóa không tồn tại.

Hướng giải bài tập bổ sung

Cây chèn tăng dần có thể thành chuỗi. Không ghi mọi thao tác BST là O(log n) nếu chưa có điều kiện về chiều cao.

Câu hỏi tự kiểm tra

1. Cây nhị phân có đặc điểm nào?
2. Trong BST không khóa trùng, khóa ở cây con trái so với nút thế nào?
3. Inorder của BST hợp lệ cho thứ tự nào?
4. Tìm kiếm BST có chi phí phụ thuộc gì?
5. Chèn tăng dần vào BST không tự cân bằng có thể gây gì?

Gợi ý đáp án và điểm cần nhớ

- Mỗi nút tối đa hai con. Giới hạn số con không tự tạo quy tắc tìm kiếm.

- Nhỏ hơn khóa nút. Quy tắc thứ tự áp dụng cho toàn cây con.
- Tăng dần. Inorder là trái–nút–phải.
- Chiều cao cây. Mỗi bước đi xuống một mức.
- Cây lệch và tìm kiếm O(n). Trường hợp xấu cây có chiều cao tỉ lệ n.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Heap và hàng đợi ưu tiên

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 5: Ưu tiên và đồ thị.
- Các bài trong chương: Heap và hàng đợi ưu tiên; Đồ thị, BFS và DFS.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Mô phỏng min-heap và phân biệt ưu tiên với FIFO.

Kiến thức nền

Hàng đợi ưu tiên lấy phần tử theo độ ưu tiên, không nhất thiết theo thứ tự đến. Min-heap nhị phân thường biểu diễn bằng mảng và hình dạng cây nhị phân hoàn chỉnh; mỗi nút cha có khóa không lớn hơn các con. Phần tử nhỏ nhất ở gốc nhưng toàn bộ mảng chưa được sắp xếp.

Với chỉ số bắt đầu từ 0, con trái của i ở 2*i+1, con phải ở 2*i+2. Chèn phần tử vào cuối rồi đưa lên để khôi phục tính heap; lấy min thay gốc bằng phần tử cuối rồi đưa xuống. Chiều cao O(log n) dẫn tới chi phí chèn/lấy min O(log n); xem min O(1).

Heap hữu ích khi liên tục cần công việc ưu tiên hoặc k phần tử nổi bật. Nếu nhiều việc bằng ưu tiên và cần giữ thứ tự đến, bổ sung bộ đếm thứ tự. Không dùng heap để thay thế mọi yêu cầu tìm kiếm theo khóa tùy ý.

1. Bất biến và chỉ số

Min-heap yêu cầu cha không lớn hơn các con, không yêu cầu toàn nhánh trái nhỏ hơn toàn nhánh phải như BST. Hình dạng hoàn chỉnh giúp biểu diễn bằng mảng. Với i>0, cha ở (i-1)//2; con ở 2*i+1 và 2*i+2 nếu các chỉ số còn trong mảng.

2. Cài đặt sử dụng thư viện chuẩn

```
import heapq

heap = []
for value in [5, 2, 8, 1]:
    heapq.heappush(heap, value)
```

```
output = [heapq.heappop(heap) for _ in range(len(heap))]
assert output == [1, 2, 5, 8]
```

Đây là minh họa API min-heap đã có từ lâu, không phụ thuộc các hàm max-heap mới. Không gọi heappop trên heap rỗng nếu chưa quy định xử lý lỗi.

3. Truy vết thao tác và độ phức tạp

Thêm 1 vào [2,5,8] đặt ở cuối rồi đổi với cha 5, sau đó với cha 2. Lấy min thay gốc bằng phần tử cuối, đưa xuống qua con nhỏ hơn để khôi phục bất biến. Chiều cao là O(log n). heapify cả mảng có thể O(n); không nhân n với log n nếu đang phân tích thuật toán heapify tuyến tính.

4. Ưu tiên bằng nhau và ứng dụng

Lưu tuple (priority, sequence, task) để giữ thứ tự đến khi priority bằng nhau và tránh so sánh trực tiếp task không có thứ tự. Cần nêu quy ước số nhỏ là ưu tiên cao hay ngược lại.

Thực hành: thêm ba việc có ưu tiên 2,1,2; mong đợi việc ưu tiên 1 trước rồi hai việc ưu tiên 2 theo sequence. Giải thích tại sao heap không tối ưu mọi yêu cầu tìm theo mã task tùy ý.

> **Bài tập vận dụng bổ sung**
>
> Mô phỏng thêm 1 vào heap [2,5,8]. Lập lịch ba công việc có ưu tiên khác nhau và giải thích quy ước khi bằng ưu tiên.

Hướng giải bài tập bổ sung

Kết quả lấy lần lượt là 1,2,5,8 nếu không thêm phần tử khác. Nêu rõ số ưu tiên nhỏ hay lớn được phục vụ trước.

Câu hỏi tự kiểm tra

1. Gốc min-heap chứa gì?
2. Mảng biểu diễn heap có luôn sắp xếp tăng dần không?
3. Con trái của chỉ số i trong heap mảng 0-based ở đâu?
4. Chèn vào heap nhị phân thường có chi phí nào?
5. Priority queue khác queue FIFO ở điểm nào?

Gợi ý đáp án và điểm cần nhớ

- Khóa nhỏ nhất. Quy tắc heap đảm bảo min tại gốc.
- Không. Quy tắc cha–con yếu hơn thứ tự tuyến tính toàn mảng.
- 2*i+1. Công thức xuất phát từ biểu diễn cây theo từng mức.
- O(log n). Phần tử có thể đi qua chiều cao heap.
- Lấy theo ưu tiên thay vì bắt buộc thứ tự đến. Tiêu chí lấy phần tử là độ ưu tiên.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Đồ thị, BFS và DFS

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 5: Ưu tiên và đồ thị.
- Các bài trong chương: Heap và hàng đợi ưu tiên; Đồ thị, BFS và DFS.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Biểu diễn đồ thị và chọn chiến lược duyệt đúng mục tiêu.

Kiến thức nền

Đồ thị gồm đỉnh và cạnh, có thể có hướng hoặc vô hướng. Danh sách kề lưu các láng giềng của mỗi đỉnh; ma trận kề dùng ô biểu diễn quan hệ từng cặp đỉnh. Với đồ thị thưa, danh sách kề thường tiết kiệm bộ nhớ hơn ma trận $O(V^2)$.

BFS dùng queue để duyệt theo lớp. Trong đồ thị không trọng số, BFS tìm số cạnh ít nhất từ đỉnh xuất phát tới các đỉnh đến được. DFS đi sâu theo nhánh, có thể dùng đệ quy hoặc stack. Cả hai cần quản lý đã thăm để tránh lặp với chu trình.

Với danh sách kề và cách triển khai chuẩn, duyệt toàn phần liên quan có chi phí $O(V+E)$. Một lần duyệt từ s không bảo đảm thăm hết đồ thị không liên thông. Để duyệt toàn bộ, có thể bắt đầu lại ở đỉnh chưa thăm. BFS thông thường không tự giải đường đi ngắn nhất theo trọng số tùy ý.

1. Đọc đồ thị từ dữ liệu

Đồ thị có hướng cần ghi cạnh theo chiều; vô hướng thường thêm quan hệ hai chiều vào danh sách kề. Số đỉnh V và cạnh E cần nhất quán với cách lưu. Khi có self-loop hoặc cạnh trùng, quy định có giữ hay loại thay vì đếm tùy ý.

2. BFS trả khoảng cách ít cạnh nhất

```
from collections import deque

def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])
    while queue:
```

```
        node = queue.popleft()
        for nxt in graph.get(node, []):
            if nxt not in distance:
                distance[nxt] = distance[node] + 1
                queue.append(nxt)
    return distance

g = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
assert bfs_distances(g, 'A')['D'] == 2
```

Đánh dấu khi phát hiện giúp một đỉnh không bị thêm lặp nhiều lần. Hàm chỉ trả các đỉnh đến được từ start.

3. DFS và lựa chọn chiến lược

DFS có thể ghi thời điểm vào/ra để hỗ trợ nhiều bài toán, nhưng một lần DFS không tự cho đường ít cạnh nhất. Đệ quy có thể vượt độ sâu khi đồ thị lớn; stack tường minh là một lựa chọn. Nếu cần đường theo trọng số, phải dùng thuật toán phù hợp điều kiện trọng số, không áp BFS thường cho mọi trường hợp.

4. Bài tập và đáp án

Thêm E cô lập: BFS từ A không có E trong kết quả. Thêm D→A tạo chu trình: hàm vẫn kết thúc vì đã có distance. Đổi thứ tự kề có thể đổi thứ tự khám phá nhưng không đổi khoảng cách ít cạnh nhất. Nộp bảng queue qua từng bước và đường đi lấy qua mảng cha được bổ sung.

> **Bài tập vận dụng bổ sung**
>
> Viết danh sách kề, vẽ vết queue BFS và stack DFS. Thêm đỉnh E cô lập rồi kiểm tra một lần BFS từ A có thăm E không.

Hướng giải bài tập bổ sung

Không thăm E nếu không có đường nối. Đánh dấu phát hiện phù hợp để tránh đưa cùng đỉnh vào queue lặp lại nhiều lần.

Câu hỏi tự kiểm tra

1. BFS thường dùng cấu trúc nào?
2. DFS có thể triển khai bằng gì?
3. BFS tìm đường ít cạnh nhất trong loại đồ thị nào?
4. Vì sao cần tập đã thăm?
5. Duyệt bằng danh sách kề thường có chi phí nào?

Gợi ý đáp án và điểm cần nhớ

- Queue. Queue tạo thứ tự theo lớp.
- Stack hoặc đệ quy. Ngăn xếp hỗ trợ quay lại sau nhánh sâu.
- Đồ thị không trọng số. Mỗi lớp BFS tương ứng thêm một cạnh.
- Tránh lặp lại do chu trình. Quản lý đã thăm giúp duyệt hữu hạn và hiệu quả.
- O(V+E). Xét đỉnh và cạnh một số lần hữu hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Chia để trị và quy hoạch động nhập môn

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 6: Thiết kế lời giải và thực hành.
- Các bài trong chương: Chia để trị và quy hoạch động nhập môn; Dự án nhỏ: bộ tra cứu và thống kê dữ liệu.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Phân biệt chia để trị với lưu trạng thái bài toán con.

Kiến thức nền

Chia để trị chia bài toán thành các phần nhỏ, giải rồi kết hợp; merge sort là ví dụ. Quy hoạch động thường phù hợp khi có các bài toán con chồng lặp và cấu trúc tối ưu thích hợp. Không phải cứ có đệ quy là cần quy hoạch động.

Một lời giải quy hoạch động cần định nghĩa trạng thái, chuyển trạng thái, trường hợp cơ sở và thứ tự tính. Với số cách bước lên n bậc, mỗi bước 1 hoặc 2 bậc, có thể đặt ways[0]=1, ways[1]=1 và ways[n]=ways[n-1]+ways[n-2]. Bộ nhớ có thể giảm nếu chỉ cần hai trạng thái trước.

Tham lam chọn quyết định cục bộ, nhưng cần chứng minh cho bài toán cụ thể. Chẳng hạn đổi tiền với mệnh giá tùy ý không bảo đảm thuật toán luôn lấy đồng lớn nhất sẽ dùng ít đồng nhất. Bắt đầu từ mô hình bài toán và ví dụ phản chứng, không chọn kỹ thuật theo tên gọi hấp dẫn.

1. Từ lời giải trực tiếp tới trạng thái

Xác định thông tin tối thiểu để mô tả bài toán con. Với số cách lên cầu thang, trạng thái chỉ cần số bậc còn lại khi bước cho phép luôn là 1 hoặc 2. Nếu luật thay đổi theo bước trước, trạng thái phải chứa thêm thông tin; không tái sử dụng công thức cũ một cách máy móc.

2. Cài đặt bottom-up có kiểm tra biên

```
def stair_ways(n):
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError('invalid n')
    previous, current = 1, 1
    for _ in range(n):
```

```
        previous, current = current, previous + current
    return previous

assert stair_ways(0) == 1
assert stair_ways(4) == 5
assert stair_ways(6) == 13
```

Theo mô hình phép toán số học hằng số: O(n) thời gian, O(1) số biến phụ. Với số nguyên rất lớn, chi phí bit và bộ nhớ số nguyên tăng theo kích thước giá trị, nên mô hình này là một giả định phân tích.

3. Phân biệt kỹ thuật

Chia để trị nhấn mạnh chia, giải và kết hợp; quy hoạch động lưu lời giải bài toán con hữu ích để tránh tính lại. Tham lam cần tính chất bảo đảm quyết định cục bộ dẫn tới tối ưu. Với mệnh giá 1,3,4 và tiền 6, lấy 4 trước cho ba đồng, còn 3+3 chỉ hai đồng.

4. Bài tập có lời giải định hướng

Lập bảng số đồng ít nhất cho tiền từ 0 tới 6 với các mệnh giá trên. Đặt dp[0]=0; trạng thái chưa đạt dùng vô cực; xét mọi đồng không vượt số tiền và lấy min(dp[s-coin]+1). Kết quả cho 6 là 2. Kiểm tra riêng trường hợp không tạo được số tiền nếu bỏ mệnh giá 1.

> ### Bài tập vận dụng bổ sung
>
> Lập bảng ways từ 0 tới 6 và viết phiên bản dùng hai biến. Tự kiểm tra giả định mỗi bước chỉ là 1 hoặc 2, không áp công thức cho yêu cầu khác.

Hướng giải bài tập bổ sung

ways[5]=8, ways[6]=13. Nêu rõ trạng thái 0 biểu diễn một cách không thực hiện bước nào, giúp công thức nhất quán.

Câu hỏi tự kiểm tra

1. Thành phần nào cần định nghĩa trong quy hoạch động?
2. Merge sort minh họa kỹ thuật nào?
3. Với ways[0]=1, ways[1]=1 và bước 1 hoặc 2, ways[4] là gì?
4. Tham lam lấy đồng lớn nhất có tối ưu cho mọi hệ mệnh giá không?
5. Nếu chỉ cần hai trạng thái trước, có thể tối ưu gì?

Gợi ý đáp án và điểm cần nhớ

- Trạng thái và công thức chuyển. Trạng thái, cơ sở và thứ tự tính quyết định lời giải.
- Chia để trị. Thuật toán chia, giải các nửa và trộn.
- 5. Dãy tiếp theo là 2,3,5.

- Không. Hệ 1,3,4 với số tiền 6 là một phản ví dụ.
- Bộ nhớ lưu bảng. Có thể lưu hai giá trị thay cho toàn bảng.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Dự án nhỏ: bộ tra cứu và thống kê dữ liệu

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 6: Thiết kế lời giải và thực hành.
- Các bài trong chương: Chia để trị và quy hoạch động nhập môn; Dự án nhỏ: bộ tra cứu và thống kê dữ liệu.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Kết hợp lựa chọn cấu trúc, kiểm thử và phân tích chi phí trong một chương trình nhỏ.

Kiến thức nền

Thiết kế một bộ tra cứu tên và tần suất xuất hiện. Yêu cầu gồm thêm dữ liệu, đếm, tìm và xuất báo cáo theo thứ tự. Tách quy tắc chuẩn hóa khỏi cấu trúc lưu: chẳng hạn có gộp chữ hoa/thường hay không phải được quyết định trước.

Map phù hợp cập nhật tần suất, set phù hợp kiểm tra duy nhất, danh sách sắp xếp phù hợp xuất thứ tự và tìm nhị phân trên bản dữ liệu đã sắp. Không tự dùng tìm nhị phân sau khi thêm dữ liệu mà không duy trì thứ tự. Nếu chỉ cần top k liên tục, cân nhắc heap và giải thích chi phí.

Sản phẩm cần tài liệu đầu vào/đầu ra, test và nhận xét Big-O cho từng chức năng. Tách dữ liệu rỗng, trùng lặp, Unicode và quy mô lớn. Benchmark phải cùng môi trường và cùng dữ liệu; không suy ra thuật toán đúng chỉ vì chạy nhanh.

1. Đặc tả sản phẩm

Đầu vào là danh sách tên; đầu ra gồm số bản ghi, số tên phân biệt, tần suất và báo cáo có thứ tự. Chọn chính sách bỏ khoảng trắng đầu/cuối và gộp hoa/thường rõ ràng. Nếu giữ nguyên tên để hiển thị, có thể lưu riêng khóa chuẩn hóa và nhãn hiển thị.

2. Chương trình mẫu có thể chạy

```
def build_report(names):
    counts = {}
    for name in names:
        key = name.strip().casefold()
        if not key:
```

```
            continue
        counts[key] = counts.get(key, 0) + 1
    return sorted(counts.items(), key=lambda item: (-item[1], item[0]))

assert build_report(['An', ' Bình ', 'an', '']) == [('an', 2), ('bình', 1)]
assert build_report([]) == []
```

Theo hợp đồng mẫu, chuỗi rỗng bị bỏ qua nên tổng tần suất bằng số mục hợp lệ, không phải số mục thô. Nếu đề bài muốn giữ mục rỗng phải thay cả cài đặt và test.

3. Phân tích và nâng cấp

Với n mục và k khóa khác nhau, đếm thường O(n) kỳ vọng theo mô hình băm; sắp xếp O(k log k), chưa tính độ dài chuỗi. Giữ dữ liệu nguồn nếu cần truy vết lỗi chuẩn hóa. Nếu báo cáo lấy top k liên tục trên luồng lớn, đề xuất heap và so sánh với sắp xếp toàn bộ.

4. Hồ sơ nghiệm thu

Nộp mã nguồn, cách chạy, hợp đồng chuẩn hóa, tối thiểu sáu test và phân tích Big-O. Test gồm rỗng, tất cả trùng, khoảng trắng, chữ hoa/thường, Unicode và hòa tần suất. Giữ thứ tự thứ hai theo tên để báo cáo xác định. Không chứng minh đúng bằng một ảnh chụp đầu ra; phải có đầu vào và kỳ vọng đối chiếu.

> ### Bài tập vận dụng bổ sung
>
> Viết chương trình nhận một danh sách từ; xuất số từ phân biệt, tần suất và danh sách theo tên. Nộp ít nhất sáu test và phân tích thời gian cập nhật, sắp xếp, tra cứu.

Hướng giải bài tập bổ sung

Kết quả đếm phải khớp tổng số phần tử. Với k khóa, xuất báo cáo sắp xếp thường O(k log k). Giữ lại test hồi quy khi sửa xử lý Unicode.

Câu hỏi tự kiểm tra

1. Cấu trúc nào phù hợp đếm tần suất theo khóa?
2. Sau khi thêm phần tử làm mất thứ tự, trước tìm nhị phân cần gì?
3. Tổng các tần suất phải bằng gì?
4. Báo cáo sắp xếp k khóa thường cần bao nhiêu thời gian với thuật toán O(k log k)?
5. Benchmark nhanh có tự chứng minh chương trình đúng không?

Gợi ý đáp án và điểm cần nhớ

- Map. Map liên kết mỗi khóa với bộ đếm.
- Khôi phục hoặc duy trì thứ tự. Tìm nhị phân yêu cầu thứ tự phù hợp.

- Tổng số phần tử đầu vào đã tính. Mỗi phần tử đầu vào đóng góp một lần vào một bộ đếm.
- O(k log k). Chi phí sắp xếp áp dụng cho số khóa phân biệt k.
- Không. Đúng đắn cần đối chiếu kết quả và yêu cầu.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html